4. Tiefer in SQL

Mit SELECT, WHERE, GROUP BY und JOIN kannst du schon sehr viel herausfinden. Manche Fragen brauchen aber mehr: eine Menge von Werten, ein Ergebnis, mit dem man weiterrechnet, oder Zeilen, die es auf der anderen Seite gar nicht gibt.

4.1 IN

Der IN-Befehl wird in der WHERE-Klausel verwendet, um zu prüfen, ob ein Wert zu einer bestimmten Menge von Werten gehört. Er ist eine übersichtliche Alternative zu mehreren OR-Bedingungen. **NOT IN** prüft das Gegenteil.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Attribut> IN (<Menge von Werten>)
```

BEISPIEL · ALLE USER, DIE IN DER SCHWEIZ ODER IN ÖSTERREICH WOHNEN

```
SELECT username, country
FROM users
WHERE country IN ('Schweiz', 'Österreich')
```

4.2 Geschachtelte Abfragen (Kapselung)

Das Ergebnis einer SQL-Abfrage ist immer eine Tabelle. Deshalb kann man eine Abfrage in eine andere hineinschreiben – in Klammern. Man nennt das eine **Unterabfrage** (oder Kapselung). Die **innere** Abfrage wird zuerst ausgeführt, mit ihrem Ergebnis rechnet dann die **äußere** Abfrage weiter.

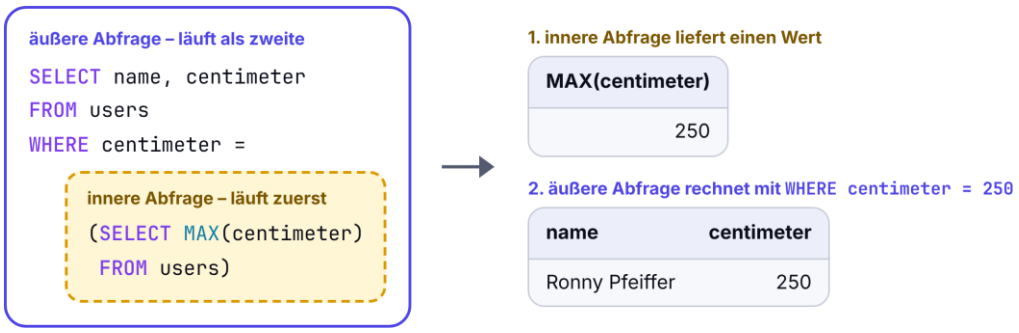


Abb. 10: Die innere Abfrage liefert den Wert, mit dem die äußere rechnet

Eine Unterabfrage kann an drei Stellen stehen. Je nachdem, wo sie steht, muss sie etwas anderes liefern:

Wo?	Was muss die Unterabfrage liefern?	Beispiel
nach `=`, `<`, `>` …	genau **einen Wert** (eine Zeile, eine Spalte)	`WHERE centimeter > (SELECT AVG(centimeter) FROM users)`
nach `IN` / `NOT IN`	eine **Liste** (eine Spalte, beliebig viele Zeilen)	`WHERE id IN (SELECT user_id FROM photos)`
nach `FROM`	eine ganze **Tabelle** – sie braucht einen Namen mit AS	`FROM (SELECT …) AS t`

EIN WERT: ALLE USER, DIE GRÖSSER ALS DER DURCHSCHNITT SIND

```
SELECT name, centimeter
FROM users
WHERE centimeter > (SELECT AVG(centimeter)
                    FROM users)
```

4.3 HAVING

WHERE filtert einzelne Datensätze **vor** der Gruppierung (GROUP BY) und entscheidet damit, welche Zeilen überhaupt in die Gruppenbildung eingehen. Aggregatfunktionen wie SUM oder COUNT sind hier nicht erlaubt.

HAVING filtert **nach** der Gruppierung und legt fest, welche gebildeten Gruppen im Ergebnis erscheinen. Dabei dürfen Aggregatfunktionen verwendet werden, da sich die Bedingung auf berechnete Gruppenwerte bezieht.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung für Zeilen>
GROUP BY <Attribut>
HAVING <Bedingung für Gruppen>
```

BEISPIEL · ALLE USER AUS KIEL MIT MEHR ALS 5 BILDERN

```
SELECT users.name, COUNT(photos.id) AS fotos
FROM users
JOIN photos ON users.id = photos.user_id
WHERE users.city = 'Kiel'
GROUP BY users.id
HAVING COUNT(photos.id) > 5
```

4.4 JOIN: mehr Varianten

Neben dem „Standard“-JOIN gibt es noch mehr Möglichkeiten, zwei Tabellen zusammenzufügen. Sie werden auch mit einem ON und deren Primär-/Fremdschlüssel verknüpft.

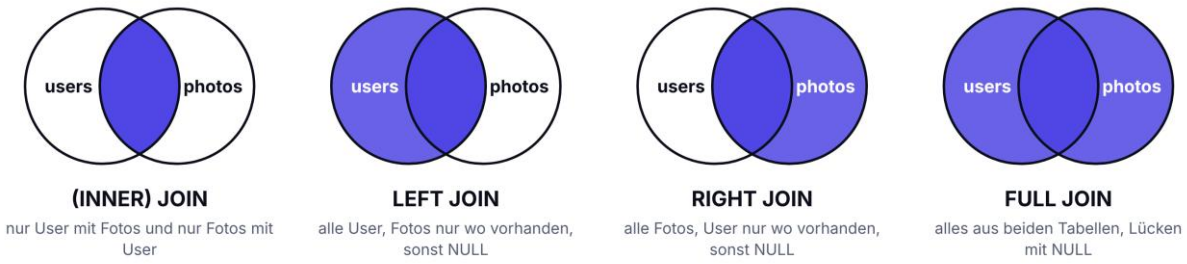


Abb. 11: Welche Zeilen bleiben bei welchem JOIN übrig?

- Der **INNER JOIN** gibt nur die Informationen zurück, die in beiden Tabellen Übereinstimmungen besitzen (INNER JOIN = JOIN).
- Der **LEFT JOIN** gibt alle Informationen der linken Tabelle (Tabelle 1) zurück, sowie die Informationen, die mit der rechten Tabelle (Tabelle 2) übereinstimmen. Gibt es keine Übereinstimmung, wird NULL ausgegeben.
- Der **RIGHT JOIN** gibt alle Informationen der rechten Tabelle (Tabelle 2) zurück, sowie die Informationen, die mit der linken Tabelle (Tabelle 1) übereinstimmen. Gibt es keine Übereinstimmung, wird NULL ausgegeben.
- Der **FULL (OUTER) JOIN** gibt alle Informationen aus beiden Tabellen aus. Sollten Werte, egal aus welcher Tabelle, keine Übereinstimmung haben, wird NULL ausgegeben.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse1>
[LEFT | RIGHT | FULL] JOIN <Klasse2>
  ON <Fremdschlüssel> =
<Primärschlüssel>
WHERE <Bedingung>
```

BEISPIEL · ALLE USER MIT IHREN BILDERN – AUCH DIE OHNE BILDER

```
SELECT users.name, photos.image
FROM users
LEFT JOIN photos ON photos.user_id = users.id
ORDER BY users.name
```

Der Trick: Wer **keinen** Partner hat, bekommt NULL. Mit `WHERE photos.id IS NULL` bleiben also genau die User ohne Foto übrig.

Aufgabe 7

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Liste alle User auf, die aus Deutschland kommen – egal, ob sie „Deutschland“, „Germany“, „deutschland“ oder „DE“ angegeben haben.

b) Gib alle User aus, die noch nie ein Foto gepostet haben. Nutze NOT IN.

c) Gib den Namen der größten Person aus – diesmal sicher richtig (vgl. Aufgabe 4 c).

d) Welche Tags wurden mindestens 8-mal verwendet?

e) Finde die 5 Fotos, die von niemandem gelikt wurden. Nutze den LEFT JOIN.

f) ★ Liste die Bilder, die nicht gelikt wurden, noch einmal auf – diesmal mit dem RIGHT JOIN. Was musst du gegenüber Aufgabe 7 e) ändern?

g) Beide Befehle sollen zählen, wie viele Fotos jeder User hat. Bei manchen Usern unterscheiden sich die Zahlen. Bei wem und warum?

BEFEHL 1

```
SELECT users.username, COUNT(*)
FROM users
LEFT JOIN photos
  ON photos.user_id = users.id
GROUP BY users.id
```

BEFEHL 2

```
SELECT users.username, COUNT(photos.id)
FROM users
LEFT JOIN photos
  ON photos.user_id = users.id
GROUP BY users.id
```

h) Einer der beiden Befehle läuft nicht. Welcher, und warum? Was musst du ändern?

BEFEHL 1

```
SELECT city, COUNT(*)
FROM users
WHERE COUNT(*) > 4
GROUP BY city
```

BEFEHL 2

```
SELECT city, COUNT(*)
FROM users
GROUP BY city
HAVING COUNT(*) > 4
```

i) ★ Welche User haben mehr Fotos gepostet als der Durchschnitt aller User, die posten?

★ zum Knobeln